

wpipe at the center of your pipelines

Is your architecture a resource devourer or an efficiency engine? 🌍

The context

I've seen architectures that need 2GB of RAM just to "exist" before processing a single byte of data. In a world crying out for Green-IT and efficiency, that's no longer acceptable. The "Architect's Awakening" happens when you realize robustness isn't measured in gigabytes consumed, but in your system's ability to stay resilient while consuming as little as possible.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="my_step", retry_count=3, retry_delay=2, checkpoint=True)
def my_step(data):
    return {"result": process(data["input"])}

pipe = Pipeline(pipeline_name="my_pipeline", tracking_db="pipeline.db")
pipe.set_steps([my_step])
pipe.run({"input": [...]})

# Python-native orchestration: checkpoints, retries, self-documenting.
```

At a glance

Aspect	Legacy / wpipe	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard

Aspect	Legacy / wpipe	wpipe
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.

Legacy: 2GB RAM
Overhead

Refactor

wpipe: <50MB RAM
Runtime

Decision Node

High Load

Recovery

Parallel Threads/Procs

SQLite Checkpoint

Resume Process

+117k downloads

<50MB RAM

SQLite WAL

A community of efficiency-first developers.

Fits containers, VMs, and Raspberry Pi-class hardware.

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **wpipe** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.